CMPSC 465 – Homework 11

Mohammad Abu Shariha

Problem 1: Unique Minimum-Cost Alignment

(i) Subproblems

Let $A = a_1 \dots a_m$ and $B = b_1 \dots b_n$. Define forward dynamic programming states:

$$F[i][j] = \text{minimum cost of aligning } A[1..i] \text{ with } B[1..j].$$

Define reverse dynamic programming states:

$$R[i][j] = \text{minimum cost of aligning } A[i+1..m] \text{ with } B[j+1..n].$$

Let $C = F[m][n]$ denote the optimal alignment cost.

(ii) Recurrence Relation and Justification

Forward DP:

$$F[i][j] = \min \begin{cases} F[i-1][j] + \text{gap}, \\ F[i][j-1] + \text{gap}, \\ F[i-1][j-1] + \text{cost}(a_i, b_j). \end{cases}$$

Reverse DP:

$$R[i][j] = \min \begin{cases} R[i+1][j] + \text{gap}, \\ R[i][j+1] + \text{gap}, \\ R[i+1][j+1] + \text{cost}(a_{i+1}, b_{j+1}). \end{cases}$$

Uniqueness Check: Traverse the proposed alignment. At each alignment state (i, j), check neighboring states (i', j') not on the submitted path. If any such transition satisfies:

$$F[i][j] + w((i, j) \to (i', j')) + R[i'][j'] = C,$$

then an alternative optimal alignment exists. If not, the alignment is unique.

Justification: F gives the minimum cost up to (i, j) and R gives the minimum cost from (i, j) to the end. If a neighboring state can still yield total cost C, then another optimal path exists.

(iii) Base Cases

Forward:

$$F[0][0] = 0, \quad F[i][0] = i \cdot \text{gap}, \quad F[0][j] = j \cdot \text{gap}.$$

Reverse:

$$R[m][n] = 0, \quad R[i][n] = (m - i) \cdot \text{gap}, \quad R[m][j] = (n - j) \cdot \text{gap}.$$

(iv) Runtime

Each table is $m \times n$ and filled in $O(1)$ per cell:

$$O(mn).$$

The uniqueness check takes $O(m + n)$. Total runtime:

$$O(mn).$$

This avoids large integer overflow issues present in counting-based approaches.

Problem 2: Speech Recognition

(i) Subproblems

Let $G = (V, E)$ be the directed graph with start vertex v_0. Let $s = \langle \sigma_1, \ldots, \sigma_k \rangle$ be the input sound sequence.

Define:

$$dp[i][v] = \text{TRUE if there exists a path from } v_0 \text{ to } v \text{ spelling } \sigma_1 \ldots \sigma_i.$$

(ii) Recurrence Relation and Justification

For each directed edge $(u, v) \in E$ labeled with $\sigma(u, v)$:

$$dp[i][v] = \text{TRUE} \quad \text{if} \quad dp[i-1][u] = \text{TRUE and } \sigma(u, v) = \sigma_i.$$

Justification: We extend all valid paths for prefix $i - 1$ by following edges whose label matches the current sound.

(iii) Base Cases

$$dp[0][v_0] = \text{TRUE}, \qquad dp[0][v] = \text{FALSE} \quad (v \neq v_0).$$

(iv) Runtime

We process k characters and check all edges:

$$O(k|E|).$$

Space usage:

$$O(k|V|).$$

If any $dp[k][v]$ is TRUE, we can reconstruct the path; otherwise return `NO-SUCH-PATH`.

Problem 3: Widest Path for All Vertex Pairs

(i) Subproblems

Let:

$$dp[k][i][j] = \text{widest path capacity from } i \text{ to } j$$

using only intermediate vertices in $\{1, \ldots, k\}$.

(ii) Recurrence Relation and Justification

$$dp[k][i][j] = \max\left(dp[k-1][i][j],\ \min(dp[k-1][i][k],\ dp[k-1][k][j])\right).$$

Justification: Either the widest path does not use vertex k, or it uses it and the bottleneck is the minimum of the two halves.

(iii) Base Cases

For $k = 0$:

$$dp[0][i][i] = \infty.$$

$$dp[0][i][j] = w(i, j) \quad \text{if edge } (i, j) \text{ exists, else } 0.$$

(iv) Runtime

Three nested loops over $|V|$:

$$O(n^3),$$

with $O(n^2)$ space.

Problem 4: Edit Distance with Transpose

(i) Subproblems

Let:

$$dp[i][j] = \text{minimum cost to transform } X[1..i] \text{ into } Y[1..j].$$

(ii) Recurrence Relation and Justification

$$dp[i][j] = \min \begin{cases} dp[i][j-1] + 1 & \text{(insert)} \\ dp[i-1][j] + 1 & \text{(delete)} \\ dp[i-1][j-1] + \begin{cases} 0 & x_i = y_j \\ 1 & x_i \neq y_j \end{cases} & \\ dp[i-2][j-2] + 1 \quad \text{if } i > 1, j > 1, x_i = y_{j-1}, x_{i-1} = y_j & \text{(transpose)} \end{cases}$$

Justification: These four operations exhaust all single-step transformations allowed.

(iii) Base Cases

$$dp[0][0] = 0, \qquad dp[i][0] = i, \qquad dp[0][j] = j.$$

(iv) Runtime

The DP table has mn entries, each computed in constant time:

$$O(mn) \text{ time}, \quad O(mn) \text{ space}.$$